

Guia Prático de Orientação a Objetos

Explorando Herança e Composição através do Projeto Fusca Azul Java v2

1. Introdução e Contexto do Projeto

No desenvolvimento de software orientado a objetos com Java, dois conceitos fundamentais ditam como estruturamos o reuso de código e a relação entre entidades: **Herança** e **Composição**. Este guia utiliza como base o projeto conceitual `fusca-azul-java-v2` para demonstrar de maneira clara, prática e elegante como aplicar essas estruturas no domínio de modelagem automobilística.

Imagine que estamos desenvolvendo um sistema para gerenciar veículos, com foco especial no icônico *Fusca Azul*. A modelagem correta impede a duplicação de lógica e garante que o sistema seja modular, escalável e de fácil manutenção.

2. O Conceito de Herança (Relação "É Um")

A herança permite que uma nova classe (subclasse) adquira os atributos e comportamentos de uma classe existente (superclasse). Ela representa uma relação de especialização, conhecida como **"É Um" (Is-A)**.

No projeto, um `Fusca` é uma especificação de um `Carro`. Logo, tudo o que um carro genérico pode fazer (como acelerar ou frear), o Fusca também pode, adicionando suas particularidades (como a cor azul clássica ou um comportamento de partida específico).

Exemplo de Implementação em Java

```
// Superclasse genérica que define as características comuns
public class Carro {
    protected String marca;
    protected String modelo;
    protected int velocidade;

    public Carro(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        this.velocidade = 0;
    }

    public void acelerar() {
        this.velocidade += 10;
        System.out.println("O carro acelerou para " + velocidade + " km/h");
    }
}

// Subclasse que herda de Carro
public class Fusca extends Carro {
    private String cor;

    public Fusca() {
        super("Volkswagen", "Fusca");
        this.cor = "Azul";
    }

    @Override
    public void acelerar() {
        // Modificação ou extensão do comportamento herdado
        this.velocidade += 8;
        System.out.println("O Fusca Azul acelerou ruidosamente para " + velocidade + " km/h");
    }
}
```

Regra de Ouro da Herança: Use apenas se a subclasse for genuinamente um tipo especializado da superclasse. Se a relação falhar no teste lógico do "É Um", a herança criará um acoplamento prejudicial.

3. O Conceito de Composição (Relação "Tem Um")

A composição consiste em projetar uma classe contendo referências a objetos de outras classes como seus membros. Em vez de herdar capacidades, a classe delega responsabilidades para os seus componentes internos. Ela representa a relação **"Tem Um"** (*Has-A*).

No caso do nosso Fusca Azul, em vez de dizer que ele *herda* de um motor, dizemos que o Fusca **tem um Motor** e **tem Rodas**. Isso confere flexibilidade total, permitindo trocar o tipo de motor sem alterar a hierarquia do carro.

Exemplo de Implementação em Java

```
// Componente independente
public class Motor {
    private int cavalos;
    private boolean ligado;

    public Motor(int cavalos) {
        this.cavalos = cavalos;
        this.ligado = false;
    }

    public void ligar() {
        this.ligado = true;
        System.out.println("Vrum! Motor ligado.");
    }
}

// Classe que compõe os elementos
public class FuscaAzulComposto {
    // Relações de Composição
    private Motor motor;
    private String placa;

    public FuscaAzulComposto(Motor motor, String placa) {
        this.motor = motor;
        this.placa = placa;
    }

    public void darPartida() {
        System.out.println("Acionando a chave do Fusca...");
        motor.ligar(); // Delegação de comportamento
    }
}
```

4. Comparativo Direto: Herança vs. Composição

A tabela a seguir resume as principais diferenças técnicas e arquiteturais ao projetar os elementos do seu ecossistema Java.

Critério	Herança (<i>Is-A</i>)	Composição (<i>Has-A</i>)
Tipo de Acoplamento	Forte (Estático, definido em tempo de compilação)	Fraco (Dinâmico, pode ser alterado em tempo de execução)
Visibilidade	Quebra o encapsulamento (subclasse vê protected)	Mantém o encapsulamento estrito dos componentes
Flexibilidade	Rígida. Alterar a superclasse afeta todas as subclasses	Alta. Componentes podem ser substituídos facilmente
Exemplo no Projeto	<code>Fusca extends Carro</code>	<code>Carro tem um Motor</code>

5. Conclusão e Melhores Práticas

O design orientado a objetos moderno favorece a **composição sobre a herança** sempre que possível. No projeto `fusca-azul-java-v2`, essa máxima fica evidente: enquanto a herança é excelente para categorizar que o Fusca pertence à família dos carros, a composição é o que permite injetar diferentes tipos de motores, rodas ou acessórios sem criar uma árvore hierárquica complexa e frágil.

Ao expandir o projeto, lembre-se: use herança para polimorfismo conceitual e reutilização de interface; use composição para estruturar funcionalidades internas modulares e flexíveis.